QUEEN'S UNIVERSITY BELFAST

GIFT City India

ITAG7105 Introduction to Data Mining and Python – I

Zaid Bin Ahsan
Lecturer

Week 1

Contents

Overview of the module objectives.
Importance of data mining and analytics.
Structured/Unstructured/Semi-structured data in organisations.

Python I: Basics, syntax, data types, conditional statements, and looping structures.

Hands-on coding: Exercises to familiarise students with Python & its environment.

QUEEN'S UNIVERSITY BELFAST

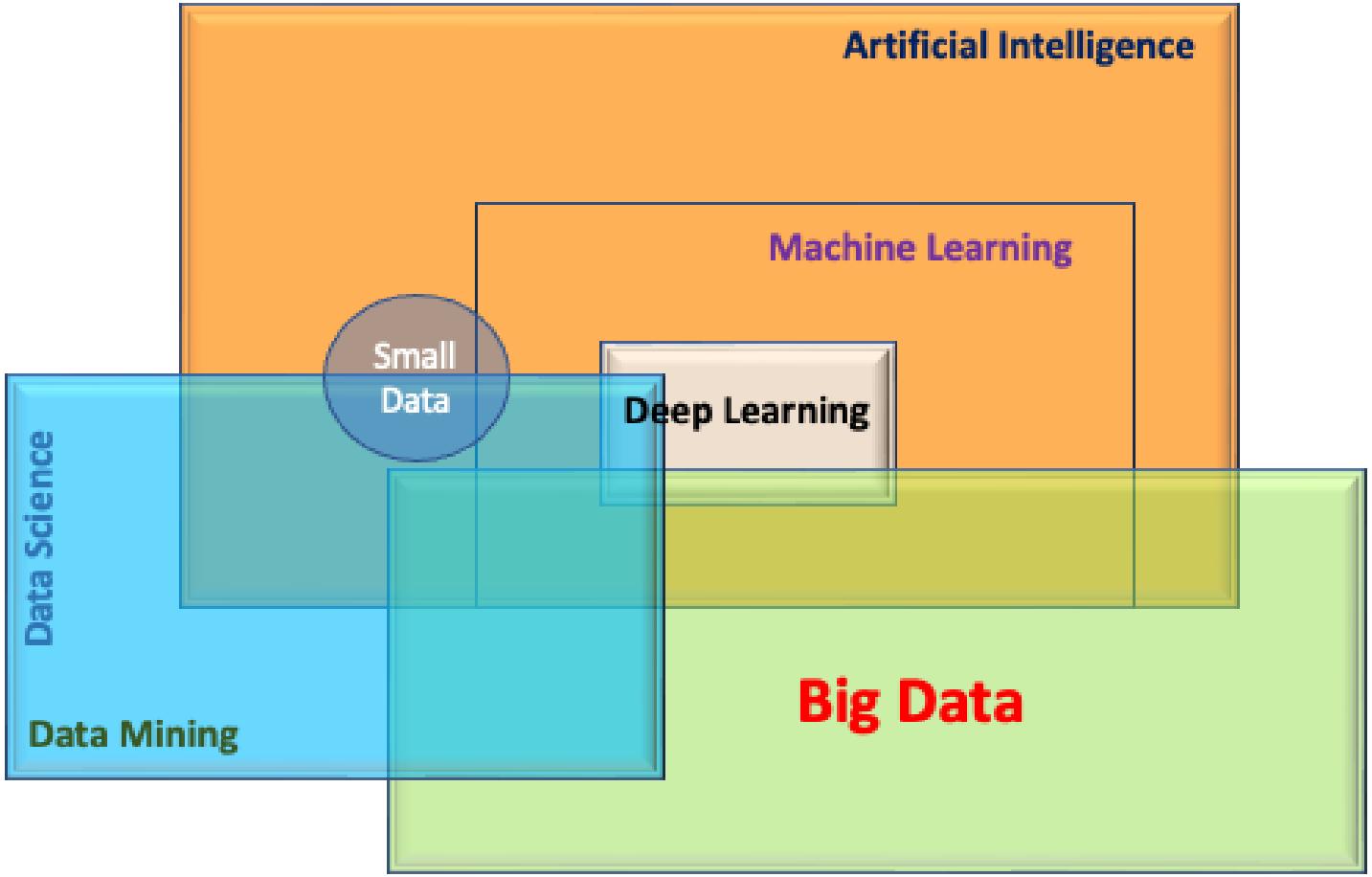
Learning Objectives

To understand the following topics:

- The fundamental concepts of data mining.
- The five types of data analytics.
- The difference between complexity and difficulty.
- The importance of a problem-centric thinking approach.
- Explore the importance of data mining in decision-making processes.
- Apply data analytics to extract meaningful insights from structured, unstructured, and semi-structured data.
- Basics of Python coding, syntax, data types, conditional statements, and looping structures.

QUEEN'S UNIVERSITY BELFAST

Understanding AI/ML/Data Science



Source: Charles, V., Garg, P., Gupta, N., & Agarwal, M. (2023). Data Analytics and Business Intelligence: Computational Frameworks, Practices, and Applications. CRC Press, Taylor and Francis Group.

Importance of Data Mining and Analytics

Data mining and analytics play a crucial role in today's business and technological landscape. Here are some key reasons why they are essential:

- **Informed Decision-Making:** Data mining allows organisations to analyse large datasets and derive valuable insights. This information aids in making informed decisions, contributing to the overall success of the business.

- **Customer Segmentation:** Through data mining, organisations can segment their customer base, enabling targeted marketing strategies and personalised customer experiences.

- **Risk Management:** Data analytics helps identify potential risks and threats, allowing organisations to proactively mitigate them. This is particularly crucial in industries such as finance and healthcare.

- **Operational Efficiency:** By analysing operational data, organisations can optimise processes, reduce inefficiencies, and enhance overall operational performance.

- **Innovation:** Data mining can uncover patterns and trends that lead to innovative solutions and new product/service developments.

QUEEN'S UNIVERSITY BELFAST

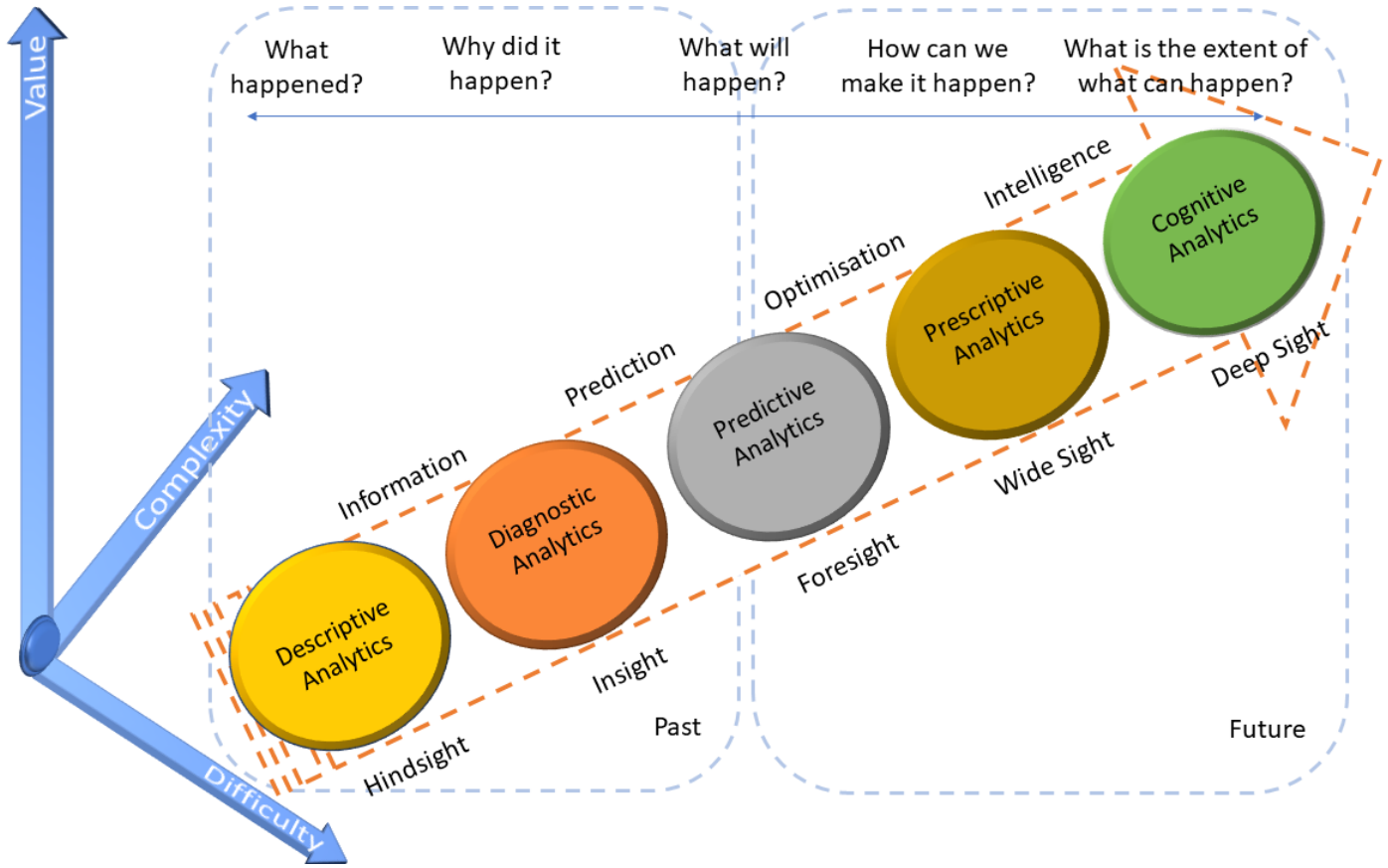
Structured/Unstructured/Semi-Structured Data in Organisations

Data Type	Description	Examples
Structured Data	Data organised in a tabular format with a clear and defined schema. Easy to query and analyse.	
Unstructured Data	Data that lacks a predefined structure and is typically in a natural language or multimedia format.	
Semi-Structured Data	Data that has a partial structure, often containing tags, keys, or elements that provide some organisation.	

QUEEN'S UNIVERSITY BELFAST

Structured/Unstructured/Semi-Structured Data in Organisations

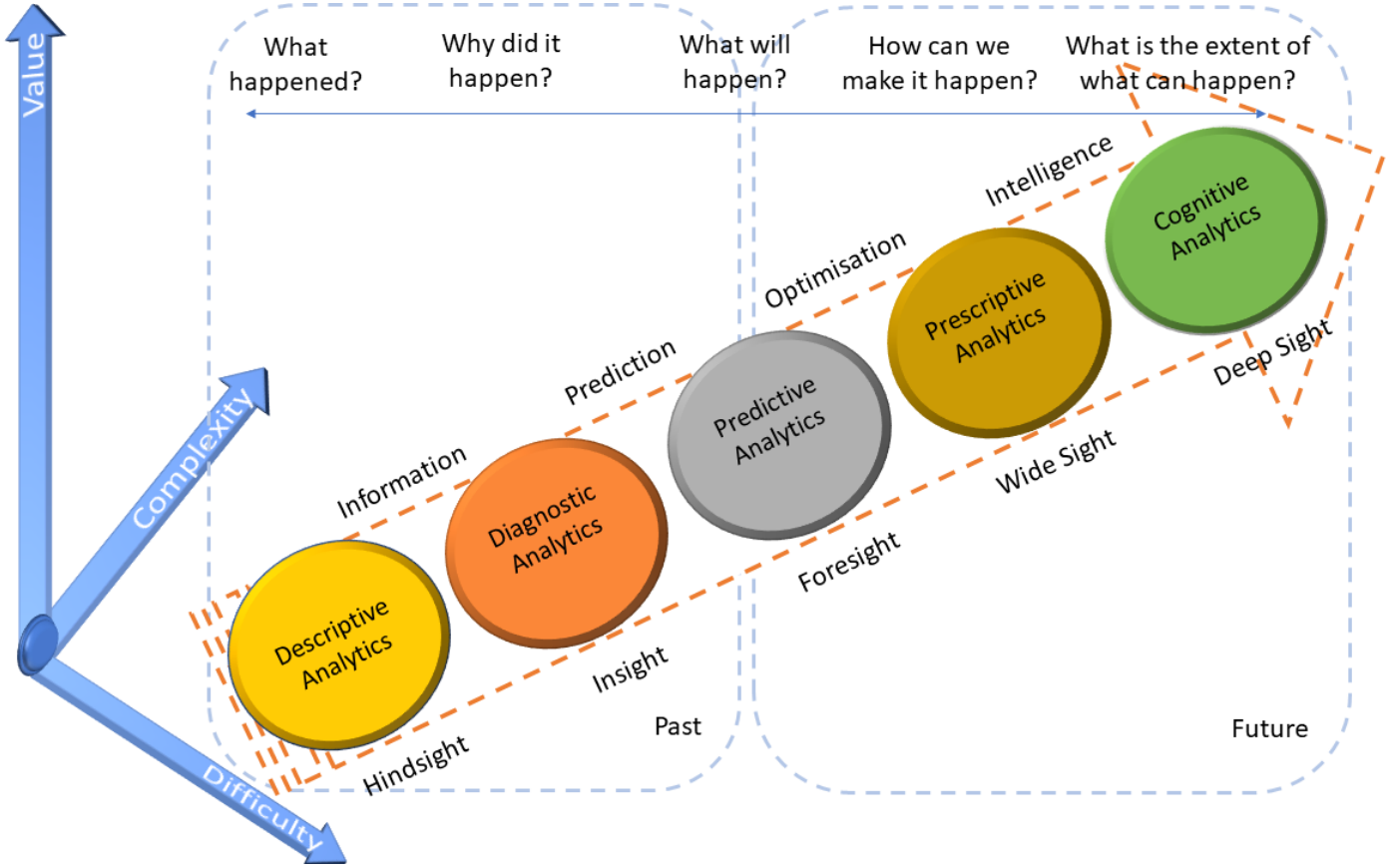
Data Type	Description	Examples
Structured Data	Data organised in a tabular format with a clear and defined schema. Easy to query and analyse.	- Relational databases (e.g., SQL databases)
		- Excel spreadsheets
		- Customer information in CRM systems
Unstructured Data	Data that lacks a predefined structure and is typically in a natural language or multimedia format.	- Text documents (Word, PDF)
		- Social media posts and comments
		- Images and videos
Semi-Structured Data	Data that has a partial structure, often containing tags, keys, or elements that provide some organisation.	- JSON or XML files
		- Log files - NoSQL databases (e.g., MongoDB) - Email headers with sender, receiver, and timestamp



Extended data analytics framework

QUEEN'S UNIVERSITY BELFAST

Complexity vs. Difficulty vs. Value



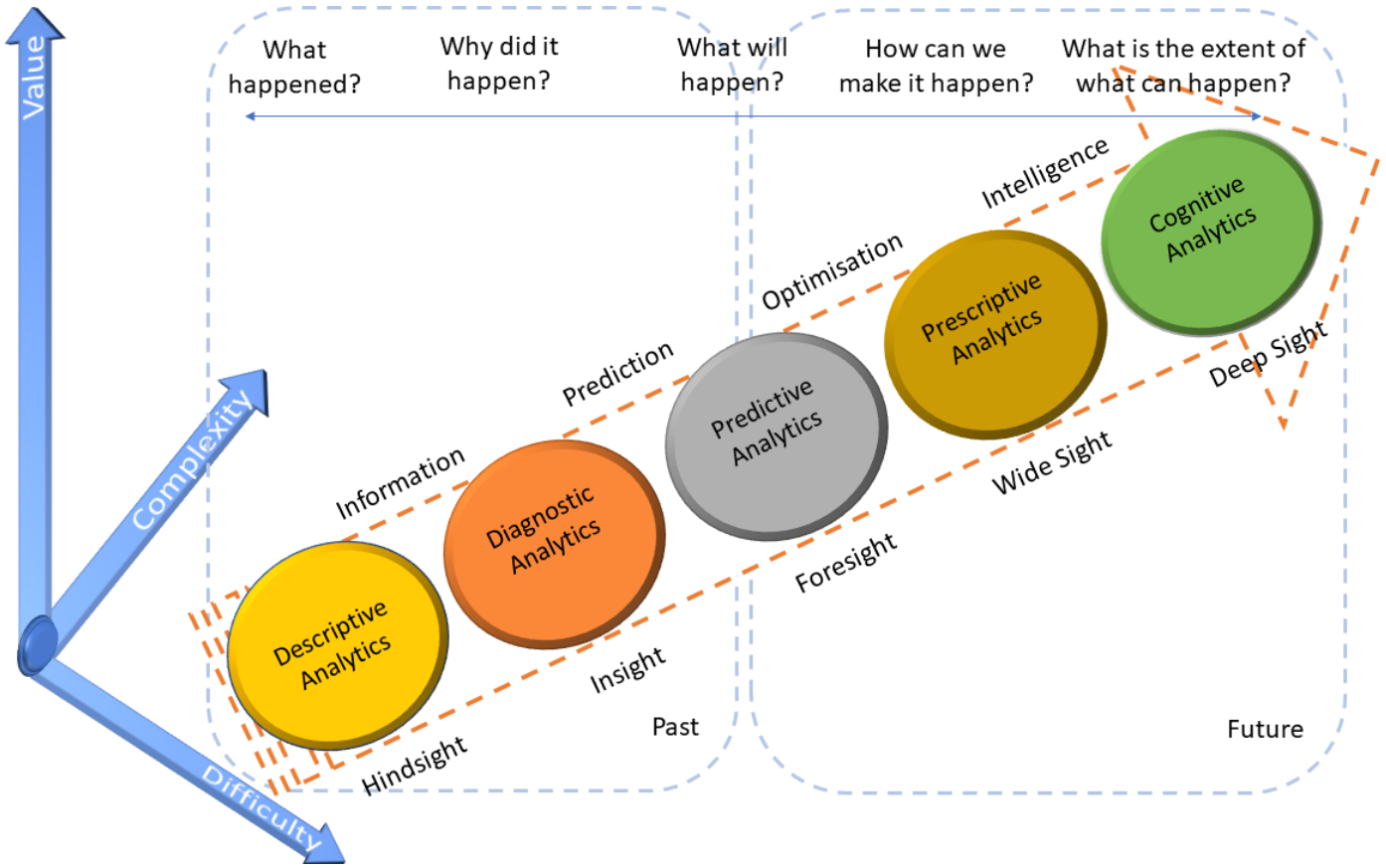
Complexity is equal to computational complexity and is a measure of the amount of computing resources (with particular focus on time and memory requirements) that algorithms consume when they run.

Complexity does not necessarily stem from large or big data.

Complexity can also arise when dealing with data for which neither the number of variables nor the number of observations is necessarily large, but for which computational need increases rapidly (i.e., the procedure to solve the problem involves a computationally intensive algorithm that requires a relatively large number of steps to complete, thus taking more time and more memory).

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707



Complexity vs. Difficulty vs. Value

Difficulty stems from the difficulty of the problem being studied and the challenges associated with extracting valuable knowledge from the data in a meaningful manner.

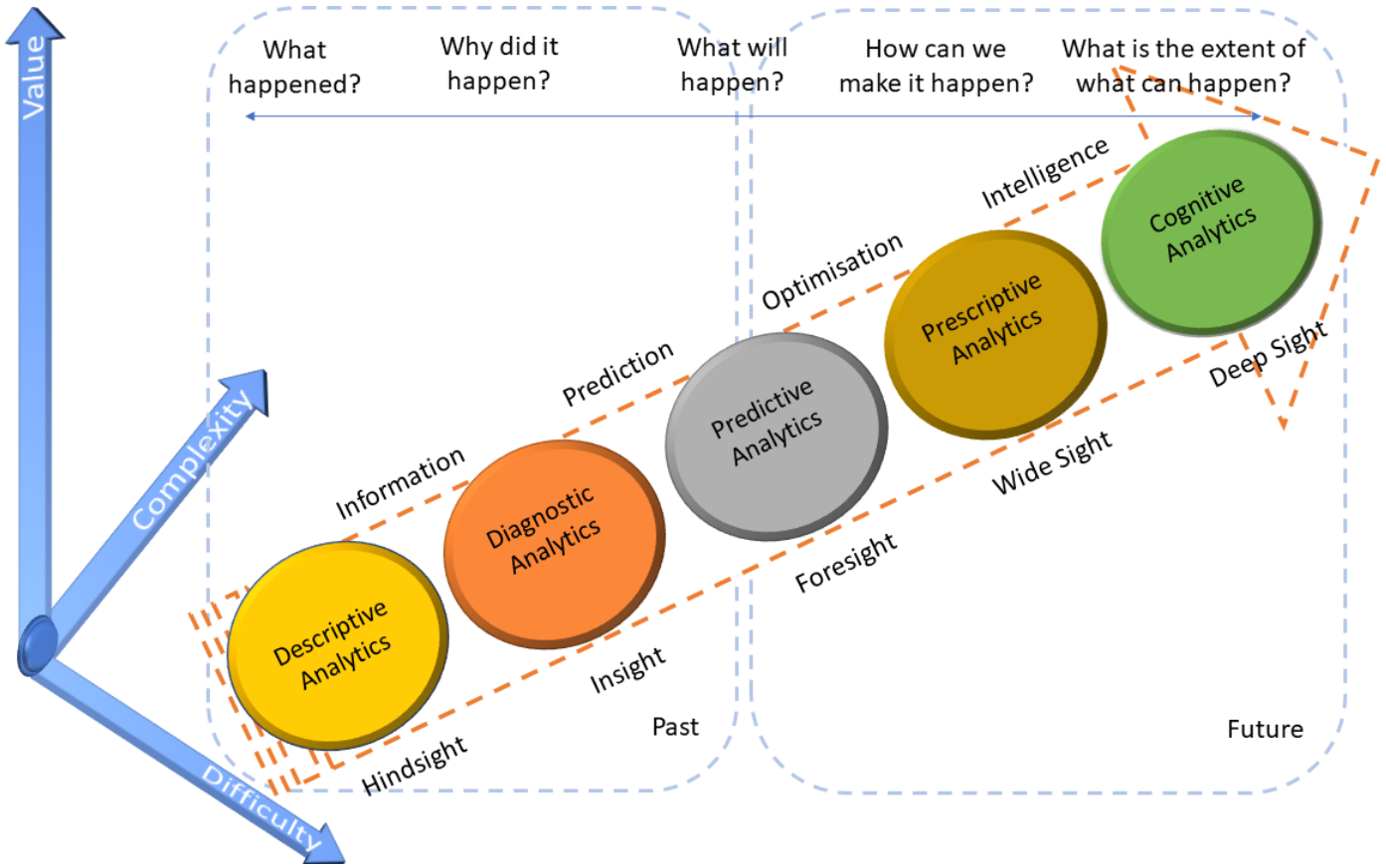
Difficult problems have many unknowns and are characterised by connectivity and mutual dependencies between involved variables, which means that, overall, these problems are very difficult to formulate.

This also means that there is no established procedure for solving difficult problems, as it is very hard to adopt or develop a rule-based approach.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Complexity vs. Difficulty vs. Value

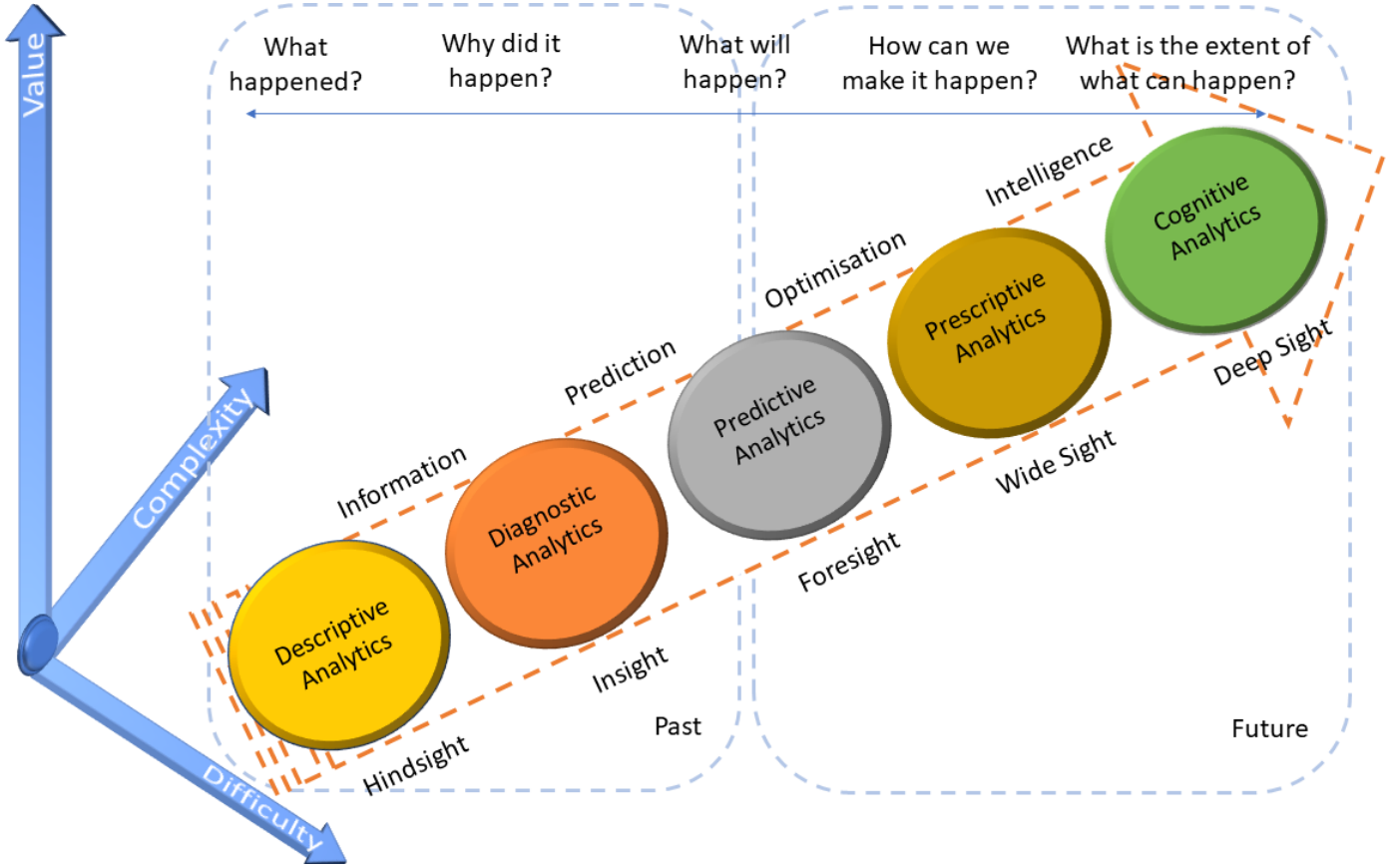


Value means various superior outcomes, such as better decision-making, improved performance, realisation of organisational objectives, and enhanced competitiveness, among others.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Descriptive analytics



It seeks to understand what phenomenon took place over a certain period of time. Hence, it focuses on summarising past data, usually in the form of dashboards.

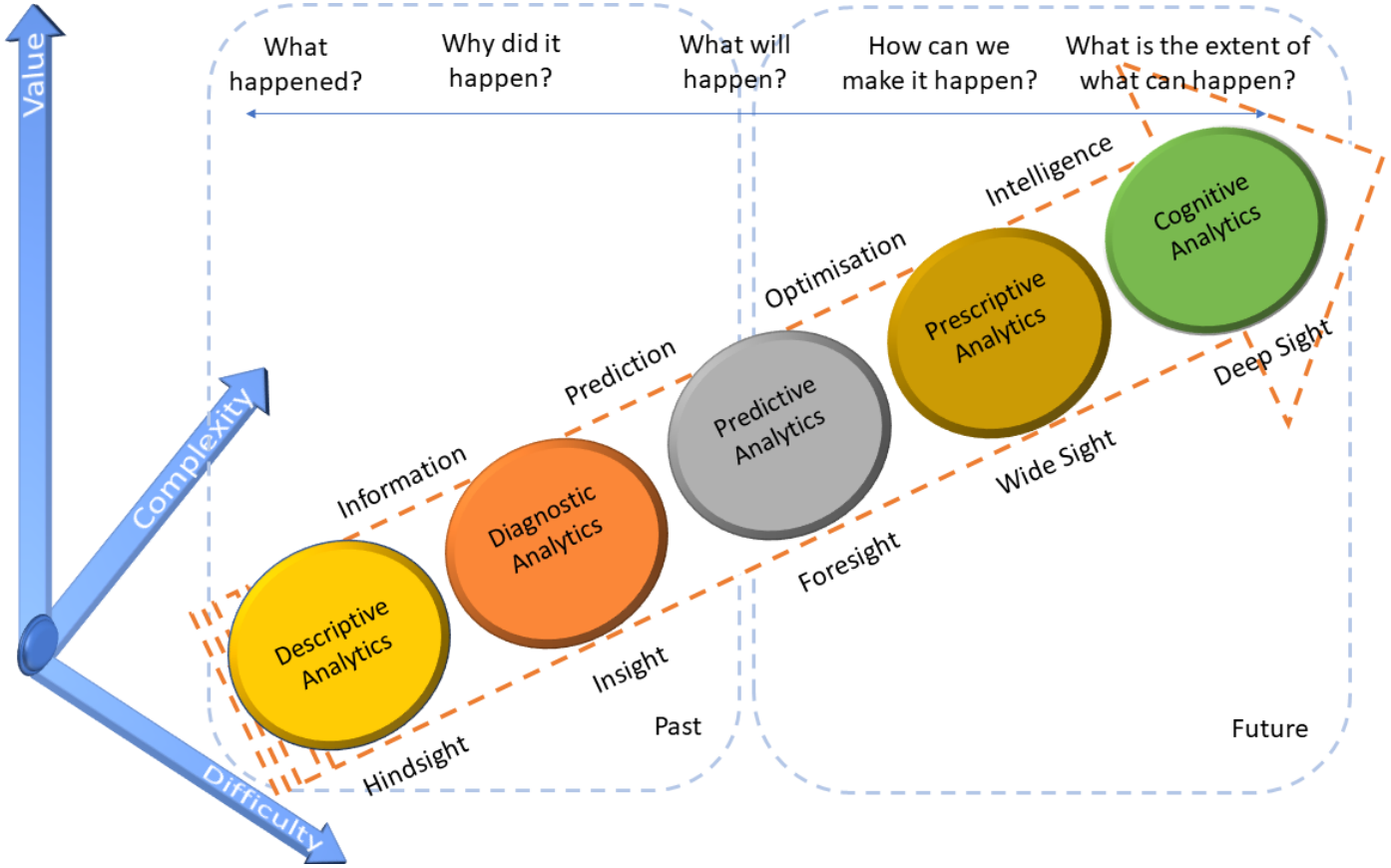
It provides *hindsight*.

Common approaches to perform descriptive analytics include visual analytics.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Diagnostic Analytics



It seeks to understand why or what caused the phenomenon to happen in the first place.

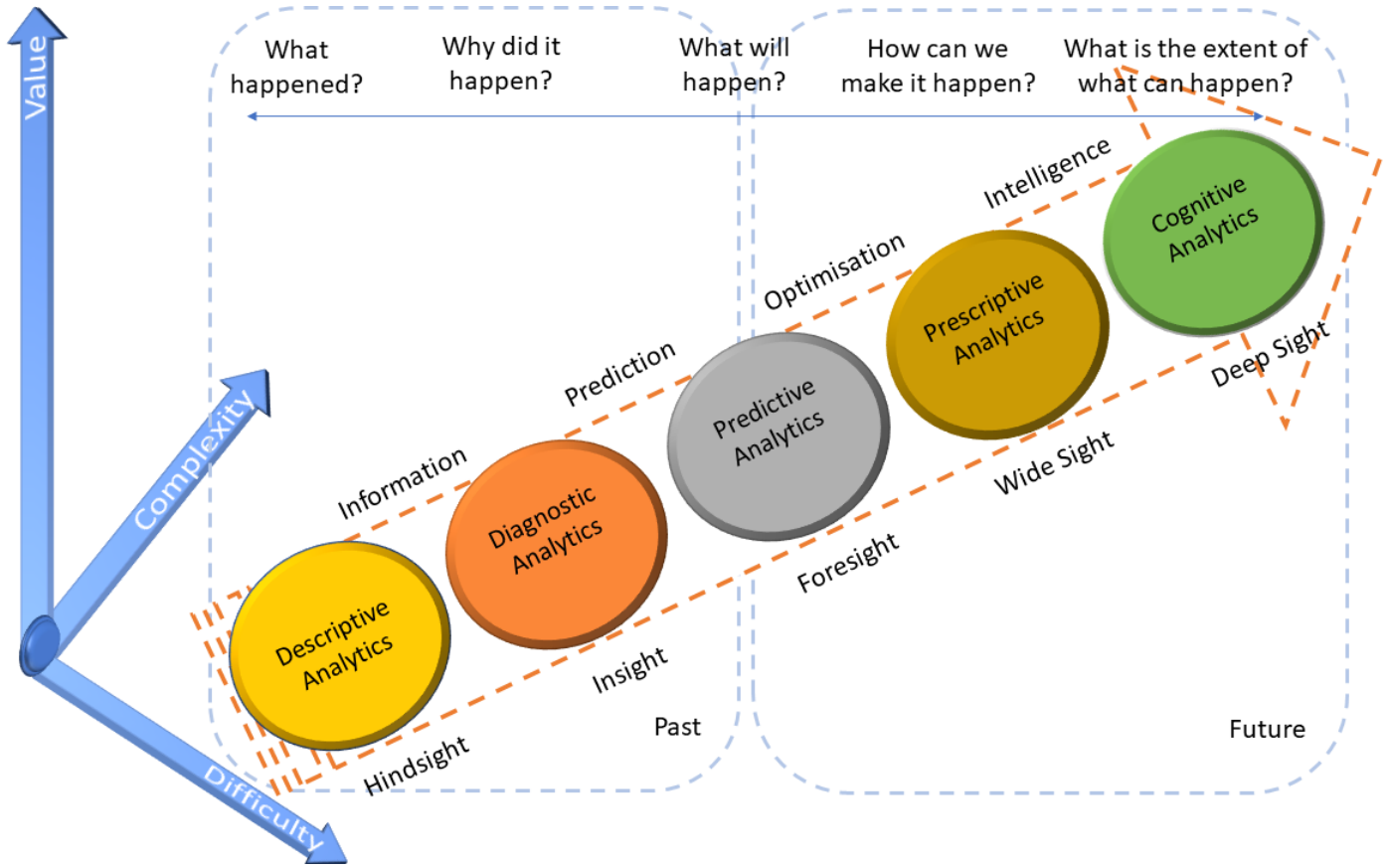
It provides *insight*.

Common approaches to perform diagnostic analytics include clustering techniques, outlier detection, Naïve Bayes, and time-series data analytics.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Predictive Analytics



It seeks to understand what can happen in the future by utilising past data to make logical predictions about future outcomes.

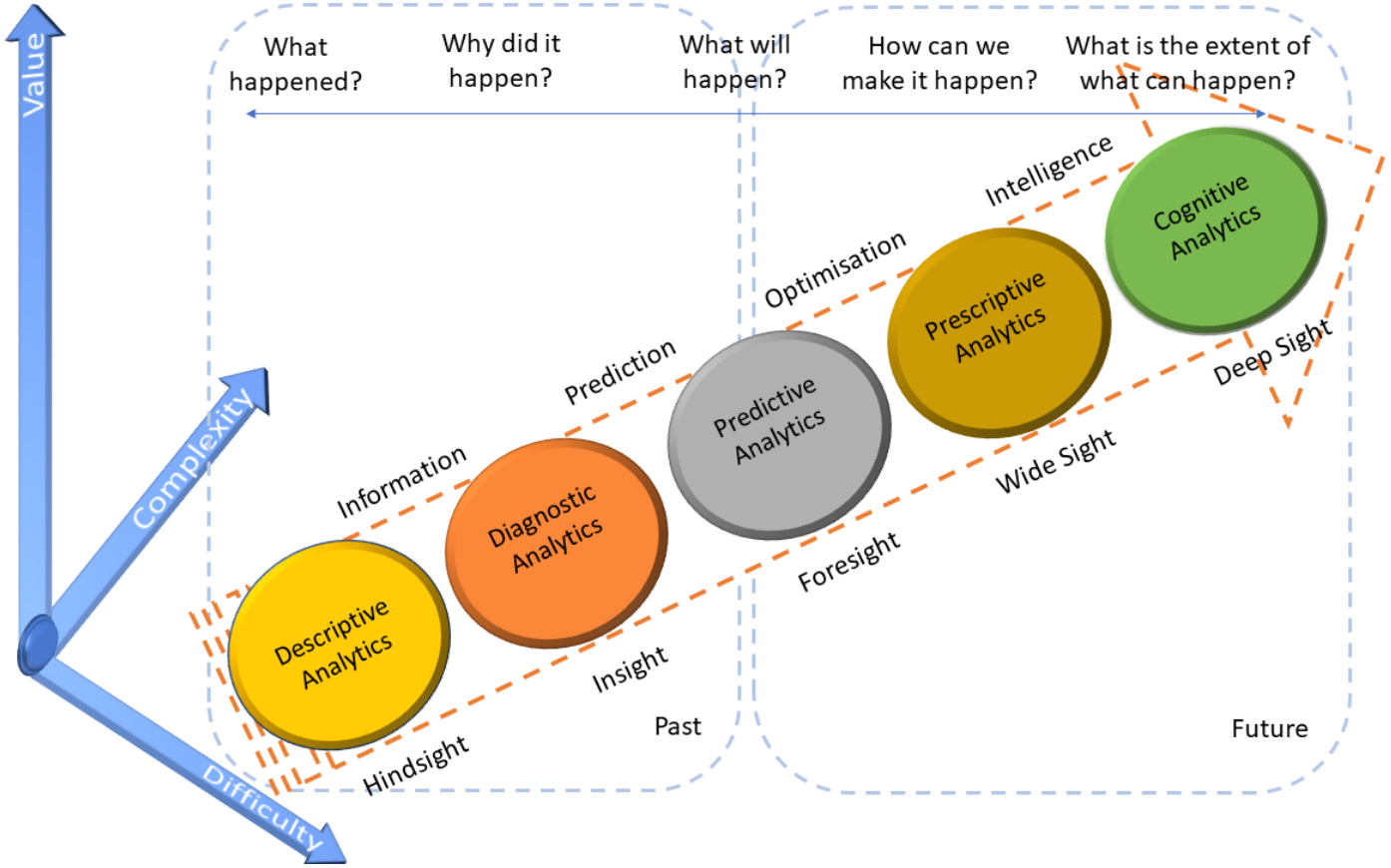
It provides *foresight*.

Common approaches to perform predictive analytics include supervised machine learning techniques, such as ensemble decision trees - random forest, support vector machines, Long Short-Term Memory (for time series prediction), and other types of artificial neural networks.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Prescriptive Analytics



It combines the knowledge obtained from all the previous analyses to predict the likely outcome of various corrective measures.

It advises organisations on all possible outcomes and the actions that are likely to optimise the desired outputs.

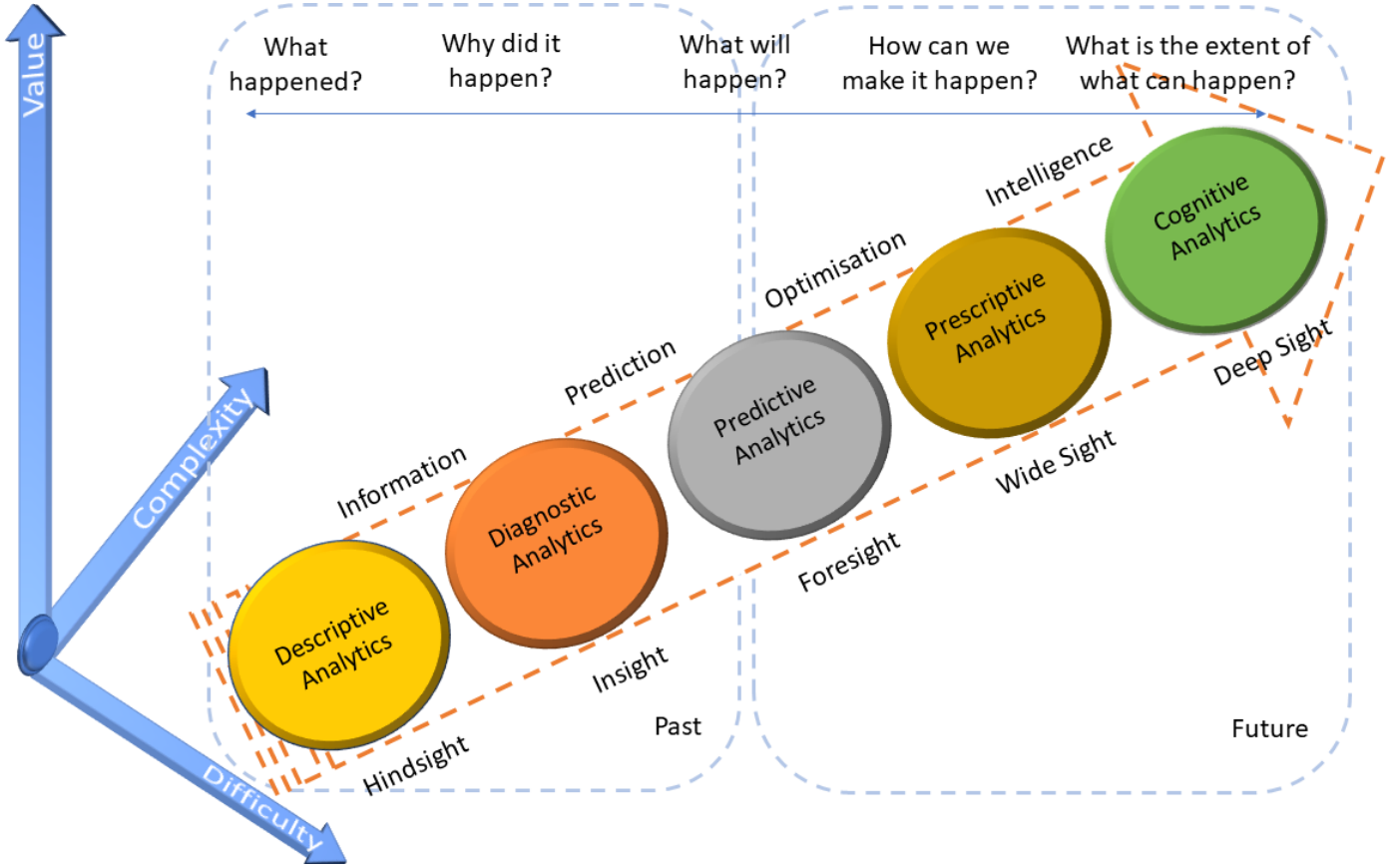
It considers and builds upon the wider context to develop a *wide sight*.

Often employs mathematical programming (such as linear, integer, and stochastic programming), game theory, Monte Carlo simulation, nature-inspired algorithms, and optimisation procedures.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Cognitive Analytics



The most advanced form of analytics, aiming to replicate human thought and mimic the way the human brain works. It is also known as “intelligent analytics”.

Helps to develop a *deep sight* of the data to unfold hidden patterns.

It works as a self-learning feedback loop, inferring from current data and then storing those inferences in the knowledge base for future inferences.

Common approaches to perform cognitive analytics include a blend of artificial intelligence (AI), machine learning (ML) algorithms (more precisely, reinforcement learning), deep learning models, semantics, and game theory.

QUEEN'S UNIVERSITY BELFAST

Source: Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. *Significance, 19*(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707

Introduction to Python

Python is a high-level, interpreted, and dynamically-typed programming language known for its readability, simplicity, and versatility. It was created by Guido van Rossum and first released in 1991.

1. **Readability and Simplicity:**
   - Python emphasises code readability and clean syntax, making it easy for developers to write and maintain code. The use of indentation instead of braces contributes to the readability.
2. **Interpreted and Interactive:**
   - Python is an interpreted language, meaning that code is executed line by line by the Python interpreter. This facilitates rapid development and debugging.
   - Python also supports an interactive mode, where you can execute code interactively in a REPL (Read-Eval-Print Loop).
3. **Versatility:**
   - Python is a general-purpose programming language with a broad range of applications, including web development, data science, artificial intelligence, machine learning, scientific computing, automation, and more.

QUEEN'S UNIVERSITY BELFAST

Introduction to Python…

4. **Extensive Standard Library:**
   Python comes with a comprehensive standard library that provides modules and packages for various functionalities. This reduces the need for external libraries in many cases.
5. **Community and Ecosystem:**
   Python has a large and active community of developers who contribute to its growth and development. The Python Package Index (PyPI) hosts a vast collection of third-party libraries and packages.
6. **Object-Oriented Programming:**
   Python supports object-oriented programming (OOP) principles, allowing developers to create and use classes and objects.
7. **Cross-Platform Compatibility:**
   Python is cross-platform, meaning that Python code can run on different operating systems with little or no modification.

QUEEN'S UNIVERSITY BELFAST

Introduction to Python…

8. **Dynamically Typed:**
   - Python is dynamically typed, which means that variable types are determined at runtime. This provides flexibility but requires careful attention to variable types during development.
9. **Community-Driven Development:**
   - The Python Enhancement Proposal (PEP) process allows the community to propose and discuss changes to the language. This democratic approach to development ensures that decisions are made collectively.
10. **Popular Frameworks:**
   - Python has numerous frameworks and libraries that facilitate the development of specific applications. Some notable ones include Django and Flask for web development, NumPy and Pandas for data science, TensorFlow and PyTorch for machine learning, and more.

QUEEN'S UNIVERSITY BELFAST

Basics of Python

Python Code: Print

```
print("Hello, Class!")
```

Python Code: Comments

```
# This is a single-line comment
```

Data Types in Python:

Data types provide flexibility for representing different kinds of information in Python, allowing for the manipulation and processing of diverse types of data in programming tasks.

QUEEN'S UNIVERSITY BELFAST

Data Types in Python…

- **Integer:**
  - Integer data type represents whole numbers without decimal points.
  - Example: x = 5
- **Float:**
  - Float data type represents numbers with decimal points.
  - Example: y = 3.14
- **Complex:**
  - Complex data type represents numbers in the form a + bj, where a and b are real numbers, and j is the imaginary unit.
  - Example: z = 1 + 2j

Python Code: Numeric Types

```
x = 5          # Integer
y = 3.14       # Float
z = 1 + 2j     # Complex
```

QUEEN'S UNIVERSITY BELFAST

Data Types in Python...

- **String:**
  - String data type represents sequences of characters and is enclosed in single or double quotes.
  - Example: message = "Hello, Class!"
- **Boolean:**
  - Boolean data type represents either True or False, often used for logical operations.
  - Example: is_true = True

Python Code: String

```
message = "Hello, Class!"
```

Python Code: Boolean

```
is_true = True
is_false = False
```

QUEEN'S UNIVERSITY BELFAST

Data Types in Python...

- **List:**
  - List data type represents an ordered collection of elements. Elements can be of different data types.
  - Example: numbers = [1, 2, 3, 4, 5]
- **Tuple:**
  - Tuple data type represents an ordered, immutable collection of elements. Once created, the elements cannot be changed.
  - Coordinates = (3, 4)
- **Dictionary:**
  - Dictionary data type represents a collection of key-value pairs. It is unordered and mutable.
  - Example: student = {"name": "John", "age": 20, "grade": "A"}

Python Code: List/Tuple/Dictionary

```
numbers = [1, 2, 3, 4, 5]  # List
coordinates = (3, 4) #Tuple
student = {"name": "John", "age": 20, "grade": "A"} #Dictionary
```

QUEEN'S UNIVERSITY BELFAST

Data Types in Python...

- **Set:**
  - Set type for unordered collections of unique elements.
  - A set in Python can hold elements of different data types, similar to a list.
  - Sets are particularly useful when you need to create a collection of unique elements, and they provide efficient membership testing. Remember that the order of elements in a set is not guaranteed, and sets do not support indexing.
  - Example: set = {1, 2, 3, 4, 5}

Python Code: set

```
set = {1, 2, 3, 4, 5}  # set
```

QUEEN'S UNIVERSITY BELFAST

Arithmetic Operators in Python :

- Addition (+)
- Subtraction(-)
- Multiplication(*)
- Division (/)
- Floor Division (//)
- Modulus (%)
- Exponentiation (**)
- Assignment with Operation (+=, -=, *=, /=, //=, %=)

Python Code: Arithmetic operators

```
a = 10
b = 3
# Basic arithmetic operations
sum_result = a + b
difference_result = a - b
product_result = a * b
quotient_result = a / b
floor_division_result = a // b
remainder_result = a % b
exponentiation_result = a ** b
a+=b
print("Sum:", sum_result)
print("Difference:", difference_result)
print("Product:", product_result)
print("Quotient:", quotient_result)
print("Floor Division:", floor_division_result)
print("Remainder:", remainder_result)
print("Exponentiation:", exponentiation_result)
print(”a=", a)
```

Conditional Statements in Python

- Conditional statements allow a programme to make decisions based on specified conditions.
- Essential for controlling the flow of a programme and executing different blocks of code based on varying conditions.

Syntax:

```
if condition:
    # Code block executed if condition is True
else:
    # Code block executed if condition is False
```

Python Code: if-else

```
x = 10
if x > 5:
    print("x is greater than 5")
else:
    print("x is not greater than 5")
```

QUEEN'S UNIVERSITY BELFAST

Conditional Statements in Python…

Syntax: if-else-if-else

```
if condition1:
    # Code block executed if condition1 is True
elif condition2:
    # Code block executed if condition2 is True
else:
    # Code block executed if No previous conditions are True
```

Python Code: if-elif-else

```
score = 75
if score >= 90:
    print("A")
elif score >= 80:
    print("B")
else:
    print("C")
```

QUEEN'S UNIVERSITY BELFAST

Logical Operators in Python

- and, or, not:
- and: True if both conditions are True.
- or: True if at least one condition is True.
- not: True if the condition is False; False if the condition is True.

Python Code: and

```
age = 25
if age > 18 and age < 30:
    print("Age is between 18 and 30")
```

Python Code: not

```
is_sunny = False

if not is_sunny:
    print("It's not sunny today.")
else:
    print("It's sunny today.")
```

QUEEN'S UNIVERSITY BELFAST

Conditional Statements in Python… (nested)

Syntax: if - (if - else) else

```
if condition1:
    # Code block executed if condition1 is True
    if condition2:
        # Code block executed if both condition1 and condition2 are True
    else:
        # Code block executed if condition1 is True but condition2 is False
else:
    # Code block executed if condition1 is False
```

Python Code: if-elif-else

```
x = 5
if x > 0:
    if x % 2 == 0:
        print("Positive and even")
    else:
        print("Positive and odd")
else:
    print("Non-positive")
```

QUEEN'S UNIVERSITY BELFAST

Ternary Operator in Python

- A concise way to write a conditional expression in a single line.

Syntax:

value_if_true if condition else value_if_false

Python Code: Basic Ternary Operator

```
age = 25
message = "Adult" if age >= 18 else "Minor"
```

Python Code: Ternary with Arithmetic Operation

```
x = 10
result = "Even" if x % 2 == 0 else "Odd"
print(result)
```

QUEEN'S UNIVERSITY BELFAST

Ternary Operator in Python

Python Code: Ternary with Print Statement

```
status = "Active"
print("User is active" if status == "Active" else "User is inactive")
```

Python Code: Nested Ternary Operators

```
x = 15
message = "Even" if x % 2 == 0 else ("Negative" if x < 0 else "Odd and Positive")
```

Python Code: Ternary for String Concatenation:

```
is_sunny = True
weather_message = "It's sunny" if is_sunny else "It's not sunny"
```

QUEEN'S UNIVERSITY BELFAST

Looping Structures in Python

- **For loop**
- While loop
- Loop control statements

Syntax:

for **variable** in **iterable:**

```
    # Code block to be executed for each iteration
    # Indentation is crucial in Python to define the scope of the
loop body
```

Python Code: for loop

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    print(num)
```

QUEEN'S UNIVERSITY BELFAST

Looping Structures in Python…

- For loop
- **While loop**
- Loop control statements

Syntax:

while **condition**:
    # Code block to be executed as long as the condition is True
    # Indentation is crucial in Python to define the scope of the loop body

Python Code: while loop

```
count = 0
while count < 5:
    print(count)
    count += 1
```

QUEEN'S UNIVERSITY BELFAST

Looping Structures in Python…

- For loop
- While loop
- **Loop control statements**

Syntax:

```
for variable in iterable:
    if condition:
        # Code block to be executed if the
        # condition is True
```

```
while condition:
    # Code block to be executed as long as the condition is True
    if additional_condition:
        # Additional code block to be executed if the additional
        #condition is True
```

Python Code: for loop with control

```
numbers = [1, 2, 3, 4, 5]
for num in numbers:
    if num % 2 == 0:
        print(f"{num} is even")
```

Python Code: while loop with control

```
count = 0
while count < 5:
    print(count)
    if count == 3:
        print("Count is 3")
    count += 1
```

Looping Structures in Python with Else statement…

In Python, the else clause in a loop is an optional block of code that gets executed when the loop condition becomes False or when the loop exhausts its iterable. Contrary to common expectations, the else branch in a loop is not executed if the loop is terminated by a break statement.

Syntax:

```
for variable in iterable:
    # Code inside the loop
else:
    # Code inside the else block (optional)
```

```
while condition:
    # Code block to be executed as long as the condition is True
else:
    # Code inside the else block (optional)
```

Python Code: for loop with else

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f"Fruit: {fruit}")
else:
    print("Loop completed. No more fruits.")
```

Python Code: while loop with else

```
count = 0

while count < 5:
    print(f"Count: {count}")
    count += 1
else:
    print("Loop completed. Count reached 5.")
```

Looping Structures in Python with Else statement…

Syntax:

```
for variable in iterable:
    # Code inside the loop
    break
else:
    # Code inside the else block (optional)
```

```
while condition:
    # Code block to be executed as long as the condition is True
    break
else:
   # Code inside the else block (optional)
```

Python Code: for loop with else

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print(f"Fruit: {fruit}")
    break
else:
    print("Loop completed. No more fruits.")
```

Python Code: while loop with else

```
count = 0

while count < 5:
    print(f"Count: {count}")
    count += 1
    break
else:
    print("Loop completed. Count reached 5.")
```

Lab Exercise: Sales Analysis (30 mints)

Objective: Analyse sales data and calculate key metrics using Python.
Dataset: Assume you have a dataset containing daily sales for a given month. The dataset is a list of integers representing daily sales figures.
sales_data = [1200, 1500, 1800, 2000, 1600, 2200, 2400, 2100, 1900, 2000, 2300, 2500, 1700, 2200, 2000, 1800, 1900, 2100, 2300, 2400, 2500, 2000, 1800, 1600, 1500]

Tasks:

1. Calculate and print the total sales for the month.
2. Determine the average daily sales and print it.
3. Identify and print the highest and lowest sales days.
4. Check if the sales on the 10th day exceed 2000; print a message accordingly.
5. Use a ternary operator to print "Good" if the total sales are above 50,000; otherwise, print “Average“.
6. Create a new list containing only the days where sales exceeded 2000.

Hints:

- Use a for loop to iterate through the sales data.
- Utilise conditional statements to check specific conditions.
- Apply arithmetic operators for calculations.
- Employ the ternary operator for concise conditional expressions.
- Functions: sum(), len(), min(), max().

QUEEN'S UNIVERSITY BELFAST

Solutions:

Task 1: Calculate and print total sales

```
sales_data = [1200, 1500, 1800, 2000, 1600, 2200, 2400, 2100, 1900, 2000,
2300, 2500, 1700, 2200, 2000, 1800, 1900, 2100, 2300, 2400, 2500, 2000,
1800, 1600, 1500]

total_sales = sum(sales_data)
print("Total Sales for the Month:", total_sales)
```

Task 2: Calculate and print average daily sales

```
average_sales = total_sales / len(sales_data)
print("Average Daily Sales:", average_sales)
```

Task 3: Identify and print highest and lowest sales days

```
highest_sales_day = max(sales_data)
lowest_sales_day = min(sales_data)
print("Highest Sales Day:", highest_sales_day)
print("Lowest Sales Day:", lowest_sales_day)
```

QUEEN'S UNIVERSITY BELFAST

Solutions:

Task 4: Check if sales on the 10th day exceed 2000

```
day_10_sales = sales_data[9]
print("Sales on the 10th Day:", day_10_sales)
if day_10_sales > 2000:
    print("Sales on the 10th day exceeded 2000.")
else:
    print("Sales on the 10th day did not exceed 2000.")
```

Task 5: Use a ternary operator to print "Good" or "Average"

```
sales_status = "Good" if total_sales > 50000 else "Average"
print("Sales Status:", sales_status)
```

Task 6: Create a new list with days where sales exceeded 2000

```
high_sales_days = [day for day in sales_data if day > 2000]
print("Days with Sales Exceeding 2000:", high_sales_days)
```

QUEEN'S UNIVERSITY BELFAST

Solutions:

Alternative way to accomplish task 6 is using append() function.

Task 6: Create a new list with days where sales exceeded 2000

```
high_sales_days = []
for day in sales_data:
    if day > 2000:
        high_sales_days.append(day)

print("Days with Sales Exceeding 2000:", high_sales_days)
```

QUEEN'S UNIVERSITY BELFAST

References

1. Charles, V, Emrouznejad, A, & Gherman, T. (2022). Two types of stories that data scientists can tell. Inside OR, 614, 16-17.
2. Charles, V., Emrouznejad, A., Gherman, T., & Cochran, J. (2022). Why data analytics is an art. Significance, 19(6), 42-45. DOI: https://doi.org/10.1111/1740-9713.01707
3. Charles, V., Garg, P., Gupta, N., & Agarwal, M. (2023). Data Analytics and Business Intelligence: Computational Frameworks, Practices, and Applications. CRC Press, Taylor and Francis Group.
4. Lutz, M. (2015). Learning Python. O'Reilly Media, Inc. Chapter 4 (Introducing Python Object Types), Chapter 5 (Numeric Types), Chapter 7 (String Fundamentals), Chapter 12 (If Tests And Syntax Rules), and Chapter 13 (While And For Loops).

QUEEN'S UNIVERSITY BELFAST